

Lab Protocol Security Exercise

Anghel Mihai-Alexandru, Stessens Klaas, Veress Armand

Table of contents

Introduction and Exercise Scenario:	2
Project scope and goals:	2
1. Procedure and Set-Up	2
1.1. Technical specifications (Initial configurations of the VM):	3
1.2. Software selection - Installing Docker	4
2. Analysis and Implementation	5
2.1. User accounts and privileges	5
2.2. Memory usage and limits	6
2.2.1. Swap memory	6
2.2.2. Docker memory limits	7
2.2.3. Storage	8
Test: Disk Exhaustion test	9
Test: Remote Code Execution test	10
Test: Memory Exhaustion test - activating OOM killer	10
2.3. Security upgrades	11
2.3.1. Operating system upgrades	11
2.3.2. Docker containers image upgrades	11
2.4. Security Frameworks	13
2.5. Log Data Extraction	15
2.6. Host Firewall	17
2.7. Brute Force Prevention (fail2ban)	18
2.8. Backend Hardening & App's Security Configurations	19
2.8.1. Fixing the most obvious mistakes	19
2.8.2. Database Layer Isolation and Docker Networking	19
2.8.3. Principle of Least Privilege & SQL Role Hardening	20
2.8.4. Media Upload Safeguards (Anti-DoS)	20
2.8.5. Rate limiter and web redirection	21
2.9. Deployment of our current version of our under-development application	21

3. Conclusions	25
References	26

Introduction and Exercise Scenario:

The team chose as the scenario **securing a Web Application Server** for hosting a **Tour Planner application** developed by the team-members in the context of a Software Engineering course.

The application is orchestrated with **docker-compose** into separate containers, featuring a **C#/ASP.net backend**, an **Angular frontend**, a **Postgres database**.

Among other things, the application supports **HTTP communication with OpenRoute-Service API** and **storing user-images** in the server's local file system.

***Note:** The initial scenario implied a Java/Spring Boot backend, but the we opted afterwards for a different technology (C#/ASP.net) due to reasons unrelated to the System Hardening protocol.

Project scope and goals:

The team's main goal is to explore the complete process of deploying and securing a web application in a real context, in order to understand the practical security implications of exposing a self-developed project to the public internet.

For context, the TourPlanner application's repository can be found at <https://github.com/Software-Engineering-2-Lab/Tour-Planner-Csharp>. We would like to note that while this application provided an interesting scenario for our team, it has been developed in parallel with writing this document and is currently still under development. Therefore, the web-app's implementation details and quality are outside the scope of this protocol. The main focus of this protocol is represented strictly by the security considerations made and the system hardening measures implemented.

1. Procedure and Set-Up

The team decided to use **Microsoft Azure Cloud Services** with a student subscription for hosting the server's virtual machine, allowing us to explore the scenario in a real context. This also means that the team members will have complete, independent access during the exercise.

1.1. Technical specifications (Initial configurations of the VM):

- **Region:** Europe (Spain Central)

** Out of the ones made available by azure: [“polandcentral”, “italynorth”, “francecentral”, “swedencentral”, “spaincentral”]

Spain Central was the closest which had a minimal sized VM available under the student subscription.

- **VM size:** Standard B2ats v2 (Monthly cost of ~\$9 USD from the student subscription credit)
- **vCPUs:** 2
- **RAM:** 1 GiB
- **Image:** Debian 12 “Bookworm” -x64 Gen2 (the most minimal choice, less extra-packages to secure)
- **OS Disk Size:** 30 GiB (Image Default)
- **Storage Tier:** Standard SSD (Locally Redundant Storage)
- **Public-IP:** 158.158.51.223

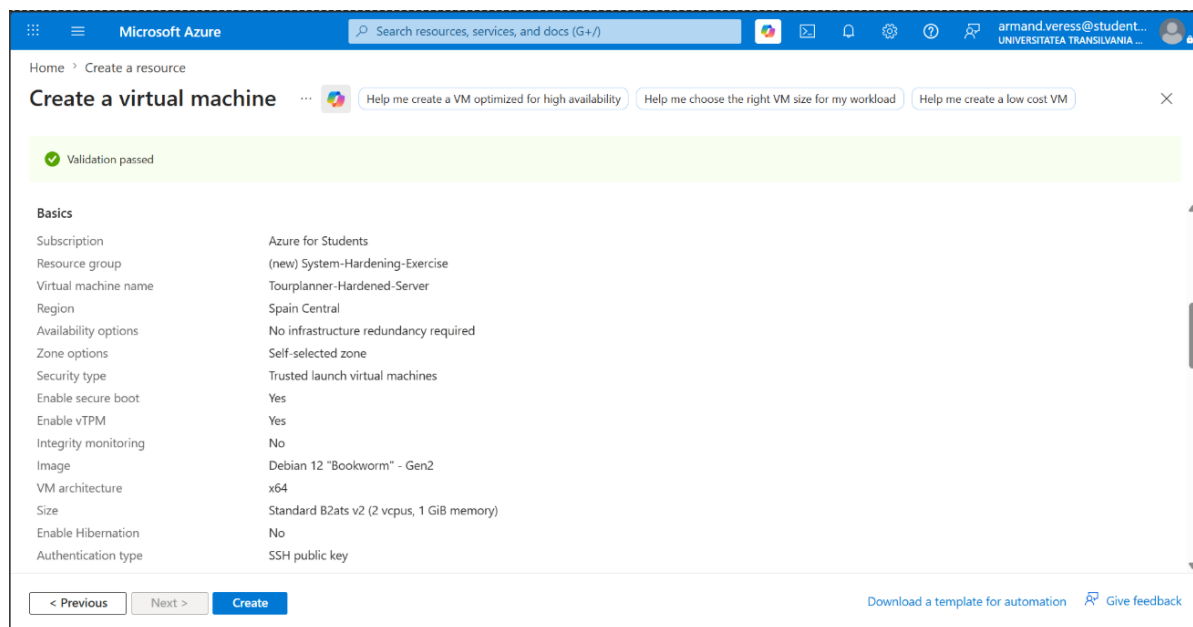


Figure 1: Figure 1. The image shows the validated overview for creating the virtual machine

1.2. Software selection - Installing Docker

The Debian “Bookworm” cloud image we chose came in our case with only 328 pre-installed packages, which ensures a minimal attack surface. All packages seemed important being for cloud integration, system operation and base security utilities.

```
~$ dpkg-query -f='${Package} ${Version} ${Architecture}\n' | wc -l
328
```

Since our web-application is orchestrated entirely with docker-compose, Docker-related packages are the only additional installations we plan. We used the official documentation for installing it using the apt repository [0]:

Setting up the Docker’s apt repository:

```
# Add Docker's official GPG key:
~$ sudo apt update
~$ sudo apt install ca-certificates curl
~$ sudo install -m 0755 -d /etc/apt/keyrings
~$ sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
~$ sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Installing the Docker packages:

```
~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose
```

Hello-world Docker verification:

```
~$ sudo docker run hello-world
...
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Docker container images:

- **Database:** *postgres:15*
- **Backend:** *mcr.microsoft.com/dotnet/aspnet:10.0*
- **Frontend:** *node:24-alpine*

2. Analysis and Implementation

2.1. User accounts and privileges

The team members proceeded with connecting with the generated azureuser admin account and a shared private key, each member creating their own user account by duplicating the generated public key in ‘.ssh/authorized_keys’ on the server.

```
~$ sudo adduser team_member
~$ sudo usermod -aG sudo team_member # grant admin privileges
~$ sudo mkdir -p /home/team_member/.ssh
~$ sudo cp /home/azureuser/.ssh/authorized_keys /home/team_member/.ssh/
~$ sudo chown -R team_member:team_member /home/team_member/.ssh # set directory ownership
~$ sudo chmod 700 /home/team_member/.ssh # secure directory access
~$ sudo chmod 600 /home/team_member/.ssh/authorized_keys #secure file access
```

This shortcut was later realized to be a mistake, as any one of us could log into each other’s accounts and so if the key were to be compromised on one device, we were all going to lose access.

This is why each team member proceeded with generating their own .ssh key locally (using the ed25519 algorithm) and updating with it the initial shared one [1].

Local

```
ssh-keygen -t ed25519
```

Virtual Machine

```
ssh -i Tourplanner-Hardened-Server_key.pem team_member@158.158.51.223 # connecting with the s
~$ nano ~/.ssh/authorized_keys # replacing the key
```

2.2. Memory usage and limits

```
~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	853Mi	328Mi	73Mi	520Ki	589Mi	524Mi
Swap:	0B	0B	0B			

Out of the 1GiB RAM of our minimal VM, we can see that the kernel and the operating system already use almost a half, leaving only 524Mb for our docker-application. Swap memory is also not configured, so this means that there is a very high chance of reaching the RAM limit without having anything to fallback to, crashing the entire server if not reglemented. Because of the high risk of activating the OOM killer, the “safety cushion” is much more significant for our server then the performance penalty [2].

2.2.1. Swap memory

Since we are using a cloud service with a running VM instance on a 30GiB partition, a repartitioning could bring complications, which is why we decided to deploy it using a swap file, allocating 2 GiB (double the RAM, respecting the standard recommendations for limited resources [2]):

```
~$ sudo dd if=/dev/zero of=/swapfile bs=1MiB count=2048
~$ sudo chmod 600 /swapfile
~$ sudo mkswap /swapfile
~$ sudo swapon /swapfile
~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab # Persist on reboot
~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	853Mi	303Mi	59Mi	496Ki	631Mi	550Mi
Swap:	2.0Gi	268Ki	2.0Gi			

In addition, to minimize the performance penalty we set the swappiness parameter to a 10 instead of the default Debian value of 60, telling our system not to fall back on the deployed swap memory untill at least 90% of the RAM is occupied [3]:

```
~$ sudo sysctl vm.swappiness=10
~$ echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
```

2.2.2. Docker memory limits

The ~550 Mi RAM also needs to be managed, setting soft limits and hard limits inside the docker-compose environment [4]. By adding these limits, we want to ensure that when RAM is at full capacity, the OOM killer will only kill our individual containers, but our system will keep its integrity.

To do this, we leave 50 Mi asside as a safe reserve and devide the rest of 500 Mi between the 3 containers accordingly: 250 Mi for our C#/ASP.net backend (200 Mi soft limit, 250 Mi hard limit), 200Mi for our Postgre database (150 soft limit, 200 hard limit) and 50 Mi for our frontend (30 Mi soft limit, 50 hard limit):

```
# docker-compose.yml
services:
  db:
    image: postgres:15
    ...
    deploy:
      resources:
        limits:
          memory: 200M
        reservations:
          memory: 150M
  backend:
    ...
    deploy:
      resources:
        limits:
          memory: 250M
        reservations:
          memory: 200M
  frontend:
    deploy:
      resources:
        limits:
          memory: 50M
        reservations:
          memory: 30M
```

2.2.3. Storage

```
~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            406M    0  406M   0% /dev
tmpfs           86M   576K   85M   1% /run
/dev/sda1       30G   3.7G   25G  13% /
tmpfs           427M    0  427M   0% /dev/shm
tmpfs           5.0M    0   5.0M   0% /run/lock
/dev/sda15      124M   12M  113M  10% /boot/efi
tmpfs           86M    0   86M   0% /run/user/1001
```

Out of the 30 GiB of memory, we have allocated 2GB for Swap memory and ~3GB is taken up by the operating system and kernel, leaving us with ~25 free GB. This means we can imagine comfortably allocating 5 GiB for the binary files of the running application, 10 GB for user-photos and the rest of 10 GB for logs and as a safety margin.

Out of these three logical sections, the storage memory for the photos is the only one likely to reach its limit, case in which we need to make sure that new uploads will be blocked, without crashing our system. By setting a limit for the photos-volume, we temporarily sacrifice a user-feature, but we keep the integrity of our system.

To prevent such a Disk Exhaustion attack, but also to enforce execution restrictions, we learned that Linux allows mount flags such as `noexec` only on actual filesystems. This led us to considering Logical Volume Management (LVM) [15], separating our disk into 2 logical partitions, one with 20GiB and one with 10GiB.

However, since our entire disk was already set up as a primary partition of 30 GB by azure, we decided not to take any risk resizing it and opted for a workaround using a `LoopDevice` [16].

Upon documentation [17], we discovered that `LoopDevices` are virtual blocks that allow files to be mounted as if they were a physical partition, and even though they are primarily meant for mounting disk images or creating encrypted containers, they are a versatile tool which we can adapt for our scenario, to avoid physically partitioning our live disk.

This approach allowed us to do exactly what we wanted initially, while also allowing the mount option `noexec` as a measure against Malware Execution.

```
~$ sudo fallocate -l 10G /opt/photos_volume.img
~$ sudo mkfs.ext4 /opt/photos_volume.img
~$ sudo mkdir -p /opt/app_folder/photos
~$ sudo mount -o loop,rw,noexec /opt/photos_volume.img /opt/app_folder/photos
~$ echo "/opt/photos_volume.img /opt/app_folder/photos ext4 loop,rw,noexec 0 0" | sudo tee -a /etc/fstab
```

```
~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            406M    0  406M   0% /dev
tmpfs           86M   580K   85M   1% /run
/dev/sda1       30G   3.8G   24G  14% /
tmpfs           427M    0  427M   0% /dev/shm
tmpfs           5.0M    0   5.0M   0% /run/lock
/dev/sda15      124M   12M  113M  10% /boot/efi
tmpfs           86M    0   86M   0% /run/user/1001
/dev/loop0      9.8G   24K   9.3G   1% /opt/app_folder/photos
```

Binding the new secured volume in docker-compose.yml:

```
services:
  ...
  backend:
    ...
    volumes:
      - /opt/app_folder/photos:/app/Storage
      - ./logs:/app/logs

volumes:
  postgres_data:
```

Test: Disk Exhaustion test

In order to test that our 10 GiB will return an error when at full capacity, protecting the rest of our storage disk, we run the following command, filling up a `exhaustion_attack.dat` file with dummy data in 1 MiB blocks using the `dd` utility. As a safety precaution, we will use `count` to set a limit of 11 GB to our writing command, ensuring that our main disk will be protected in case something unexpected goes wrong (1GB = 1024MB -> 11GB = 11264MB) [19]:

```
~$ sudo dd if=/dev/zero of=/opt/app_folder/photos/exhaustion_attack.dat bs=1M count=11264
```

The result was successful, returning the following error when full capacity was reached:

```
dd: error writing '/opt/app_folder/photos/exhaustion_attack.dat': No space left on device
9964+0 records in
9963+0 records out
10447216640 bytes (10 GB, 9.7 GiB) copied, 246.204 s, 42.4 MB/s
```

```
~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            406M    0  406M   0% /dev
tmpfs           86M   588K   85M   1% /run
/dev/sda1       30G   14G   15G  49% /
tmpfs           427M    0  427M   0% /dev/shm
tmpfs           5.0M    0   5.0M   0% /run/lock
/dev/sda15      124M   12M  113M  10% /boot/efi
/dev/loop0      9.8G   9.8G    0 100% /opt/app_folder/photos
tmpfs           86M    0   86M   0% /run/user/1001
```

Test: Remote Code Execution test

In order to test our `noexec` restriction, we created a simple shell script file and tried running it with full permissions inside our LoopDevice:

```
~$ sudo bash -c 'echo "echo \"Test failed: system compromised\"" > /opt/app_folder/photos/malware.sh'
~$ sudo chmod +x /opt/app_folder/photos/malware.sh
~$ /opt/app_folder/photos/malware.sh
-bash: /opt/app_folder/photos/malware.sh: Permission denied
```

The result was successful, returning `Permission denied` after being intercepted by the kernel.

Test: Memory Exhaustion test - activating OOM killer

In order to test our soft and hard limits, we created a simple mock up of our backend container and ran inside a simple exponential bash script:

`docker-compose-test.yml`:

```
services:
  backend-test:
    image: mcr.microsoft.com/dotnet/aspnet:10.0
    command: tail -f /dev/null
    deploy:
      resources:
        limits:
          memory: 250M
        reservations:
          memory: 200M
```

```

~$ sudo docker compose -f docker-compose-test.yml up -d
~$ sudo docker exec armand-backend-test-1 bash -c 'a="x"; while true; do a=$a$a; done'
~$ sudo docker inspect armand-backend-test-1 | grep OOMKilled
      "OOMKilled": true,
~$ free -h

```

	total	used	free	shared	buff/cache	available
Mem:	853Mi	462Mi	281Mi	520Ki	248Mi	390Mi
Swap:	2.0Gi	24Mi	2.0Gi			

The result was successful, the docker process was terminated by the OOM killer, leaving the rest of the VM's machine untouched.

2.3. Security upgrades

2.3.1. Operating system upgrades

In order to implement automatic updates for the operating system we used the pre-installed package **unattended-upgrades**. Setting the `--priority` parameter to `low` ensures that all available configuration are displayed and not only the high-priority ones.

```

~$ sudo dpkg-reconfigure --priority=low unattended-upgrades
~$ sudo tail -f /var/log/unattended-upgrades/unattended-upgrades.log
...
2026-06-08 06:33:26,581 INFO No packages found that can be upgraded unattended and no pending

```

2.3.2. Docker containers image upgrades

In order to automate the security upgrades for our container-based installation we considered tools such as Watchtower and What's Up Docker?, but opted in the end for an automated script prototype for better control and a minimal approach. We used **Cron** and created a script inside the `cron.daily` folder to be executed every 24 hours. The script updates blindly all containers where a newer version has been found, so in order to prevent unexpected discrepancy errors between versions, we control this by adding the major-version of our image-tags (*postgres:15*, *asp.net:10*, *node:24-alpine*), leaving major updates to be done manually [13]:

Creating the script:

```

~$ cd /etc/cron.daily
/etc/cron.daily$ sudo nano docker-compose-update

```

```
#!/bin/sh
set -e
echo "Commencing docker compose update $(date)" >> /var/log/docker/docker-compose-update.log
/usr/bin/docker compose -f /opt/app_folder/docker-compose.yml pull >> /var/log/docker/docker-compose-update.log
/usr/bin/docker compose -f /opt/app_folder/docker-compose.yml up -d >> /var/log/docker/docker-compose-update.log
/bin/sleep 10
/usr/bin/docker compose -f /opt/app_folder/docker-compose.yml restart >> /var/log/docker/docker-compose-update.log
echo "Commencing docker compose prune $(date)" >> /var/log/docker/docker-compose-update.log
/usr/bin/docker image prune -a -f >> /var/log/docker/docker-compose-update.log 2>&1
/usr/bin/docker container prune -f >> /var/log/docker/docker-compose-update.log 2>&1
echo "Finishing docker compose update $(date)" >> /var/log/docker/docker-compose-update.log
```

Setting up permissions to execute under root:

```
/etc/cron.daily$ sudo chmod 755 docker-compose-update
/etc/cron.daily$ sudo chown root:root docker-compose-update
```

Testing script with a running instance of a minimal `nginx:alpine` image:

```
sudo mkdir -p /opt/app_folder
sudo nano /opt/app_folder/docker-compose.yml
```

`docker-compose.yml`:

```
services:
  test_web:
    image: nginx:alpine
    ports:
      - "8080:80"
```

```
~$ cd /opt/app_folder
/opt/app_folder$ sudo /usr/bin/docker compose up -d
[+] up 2/2
    Network app_folder_default      Created
    Container app_folder-test_web-1 Started
```

*Note: Before achieving the successful start up from above we came across the following error: `Failed to Setup IP tables: Unable to enable ACCEPT OUTGOING rule:...`; Without being sure the cause of this, we tried an easy fix suggested in a forum - **restarting Docker**, which worked and led us to the conclusion that when activated, the firewall deleted the iptables and the routing used by Docker.

```
~$ sudo systemctl restart docker
```

Running script:

```
/opt/app_folder$ sudo /etc/cron.daily/docker-compose-update
/opt/app_folder$ sudo cat /var/log/docker/docker-compose-update.log
...
Commencing docker compose update Sat Jun 13 16:21:56 UTC 2026
Image nginx:alpine Pulling
Image nginx:alpine Pulled
Container app_folder-test_web-1 Running
Container app_folder-test_web-1 Restarting
Container app_folder-test_web-1 Started
Commencing docker compose prune Sat Jun 13 16:22:10 UTC 2026
Total reclaimed space: 0B
Deleted Containers:
79d7f72a1ed0ce56ccb837e635f16c0fe2bcd05d7440a7caa51a54b0a55376f0

Total reclaimed space: 4.096kB
Finishing docker compose update Sat Jun 13 16:22:10 UTC 2026
```

2.4. Security Frameworks

Our Debian 12 “Bookworm” came with AppArmor pre-installed as a native security framework for the kernel. To verify if it is active we inspected the current state [7]:

```
~$ cat /sys/module/apparmor/parameters/enabled
Y
~$ sudo aa-status
apparmor module is loaded.
12 profiles are loaded.
12 profiles are in enforce mode.
...
~$ ps auxZ | grep -v '^unconfined'
```

LABEL	USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START
/usr/sbin/chronyd (enforce)	_chrony	557	0.0	0.2	10696	2368	?	S	May24
/usr/sbin/chronyd (enforce)	_chrony	558	0.0	0.0	10564	552	?	S	May24

The AppArmor module is already compiled and enabled at the kernel level, with 2 base system services being successfully confined by the 12 enforced default profiles. Moreover, upon

documentation we have found out that when installing Docker, it automatically generates a `docker-default` profile for containers [9]. Since the 12 system profiles function properly and the `docker-default` profile will be applied automatically on all containers of our application, we decided not to be adventurous and keep these without changes.

Instead, an area where we thought exploration was in order is writing a new profile for our Log Data Extraction Service. To build this profile we used the `apparmor-utils` package, which provides the `aa-genprof` and `aa-logprof` tools for generating profiles from observed application behaviour:

```
~$ sudo apt install apparmor-utils
~$ sudo aa-status | grep rsyslog
```

At this point, `rsyslog` had no AppArmor profile of its own, confirming it was a suitable candidate for exploration. We chose `rsyslog` as our log extraction service since it provides native TCP/UDP log forwarding [10], which we use to centralise our application's container logs (see Section 6, Log Data Extraction).

Running `aa-genprof` initially failed because the minimal Debian image does not generate `/var/log/syslog` by default:

```
~$ sudo aa-genprof rsyslogd
ERROR: Can't find system log "/var/log/syslog". Please check permissions.
```

Investigation showed that while `rsyslog` was already installed and running, the cloud image runs it without a dedicated `syslog` system user (`rsyslogd` runs as `root`), and the default `/var/log/syslog` rule appeared to be inactive. We installed `rsyslog` properly via `apt` and ensured it was enabled:

```
~$ sudo apt install rsyslog
~$ sudo systemctl enable --now rsyslog
```

With `/var/log/syslog` now present, `aa-genprof rsyslogd` started successfully, placing `rsyslog` in complain mode and beginning to log its behaviour. We then exercised the service (restarts, `logger` test messages) and scanned the resulting log entries. `aa-logprof` presented each access individually for approval:

- **Capability `syslog`** — required for `rsyslog` to read kernel/system log sources. Allowed.
- **Capability `net_admin`** — required for socket option handling when forwarding logs over the network. Allowed.
- **`/etc/rsyslog.conf` (owner, read)** — `rsyslog`'s own configuration file. Allowed.
- **`/etc/nsswitch.conf`, `/etc/passwd`, `/etc/group` (owner, read)** — used via `glibc`'s NSS for `hostname/user` resolution when logging. Allowed.

After this first pass, the profile was saved and automatically loaded in **enforce** mode. Before continuing our log-extraction configuration (Section X), we switched it back to complain mode to avoid blocking new, not-yet-profiled behaviour:

```
~$ sudo aa-complain /usr/sbin/rsyslogd
~$ sudo aa-status | grep -A1 complain
```

We then configured rsyslog's network listeners and our Docker log routing (detailed in Section 6), and generated test traffic through them. A second **aa-logprof** pass captured the additional file accesses needed for writing to `/var/log/docker/containers.log` and handling the `imudp/imtcp` socket listeners. Once satisfied the profile covered normal operation, we re-enabled enforcement:

```
~$ sudo aa-enforce /usr/sbin/rsyslogd
```

Final verification confirmed the profile is active with no policy violations:

```
~$ sudo aa-status
...
14 profiles are loaded.
14 profiles are in enforce mode.
...
  /usr/sbin/rsyslogd
...
```

This gives us a custom, least-privilege AppArmor profile for our chosen log-extraction service, confining it to exactly the capabilities and files required for its operation, on top of the 12 default system profiles already enforced.

2.5. Log Data Extraction

To fulfill the requirement of extracting/securing log data, we configured **rsyslog** to listen for log messages over the network and route them into a separate, access-restricted log file. We chose to bind the listeners to `localhost` only, since our setup is a single server and there is no need for this to be reachable from outside the VM [10].

```
~$ sudo nano /etc/rsyslog.conf
```

```
module(load="imudp")
input(type="imudp" port="514" address="127.0.0.1")

module(load="imtcp")
input(type="imtcp" port="514" address="127.0.0.1")
```

We then created a dedicated directory for our application/container logs, owned by `root:adm` (this image runs `rsyslogd` as `root`, no separate `syslog` user exists), restricting read access to the `adm` group:

```
~$ sudo mkdir -p /var/log/docker
~$ sudo chown root:adm /var/log/docker
~$ sudo chmod 750 /var/log/docker
```

A routing rule sends any message tagged as coming from Docker into this file instead of the general system log:

```
~$ sudo nano /etc/rsyslog.d/30-docker.conf
```

```
if $programname startswith 'docker' then /var/log/docker/containers.log
& stop
```

```
~$ sudo systemctl restart rsyslog
```

We tested the pipeline with `logger`, both via the local socket and via the UDP listener:

```
~$ logger -t docker/testcontainer "test log message"
~$ logger -t docker/test -n 127.0.0.1 -P 514 -d "test via udp socket"
~$ sudo tail -5 /var/log/docker/containers.log
```

Both messages showed up correctly in `/var/log/docker/containers.log`, confirming the setup works end-to-end and under AppArmor enforcement (Section 5) without issues.

Once the application is deployed, each container can forward its logs to this same listener using Docker's built-in `syslog` logging driver:

```
# docker-compose.yml
services:
  backend:
    ...
```

```
logging:
  driver: syslog
  options:
    syslog-address: "udp://127.0.0.1:514"
    tag: "docker/{.Name}"
```

2.6. Host Firewall

As a defense-in-depth measure alongside the Azure Network Security Group, we set up a host firewall using `firewalld` [11].

```
~$ sudo apt install firewalld -y
~$ sudo systemctl enable --now firewalld

~$ sudo firewall-cmd --state
running

~$ sudo firewall-cmd --get-default-zone
public

~$ sudo firewall-cmd --list-all
public
  target: default
  icmp-block-inversion: no
  interfaces:
  sources:
  services: dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

The default `public` zone already covered what we needed: `target: default` rejects all unsolicited incoming traffic, `ssh` is allowed (keeping our key-based access working), `dhcpv6-client` is required for the VM's networking, and `forward: yes` is needed for Docker's bridge network once the application is deployed. We confirmed the firewall didn't break access by opening a new SSH session while keeping the old one open.

2.7. Brute Force Prevention (fail2ban)

Since our VM has a public IP and is constantly scanned, we installed **fail2ban** to automatically block IPs after repeated failed SSH logins, integrating it with **firewalld** so both tools share one set of rules [12].

```
~$ sudo apt install fail2ban
~$ sudo nano /etc/fail2ban/jail.local
```

```
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 4
banaction = firewallcmd-rich-rules

[sshd]
enabled  = true
port     = ssh
logpath  = %(sshd_log)s
backend  = systemd
banaction = firewallcmd-rich-rules
action   = %(action_)s
```

We chose 4 failed attempts within 10 minutes to trigger a 1-hour ban, and **backend = systemd** since logins are logged via **journald**. **banaction = firewallcmd-rich-rules** makes fail2ban ban IPs through **firewalld** instead of its own separate **iptables** rules — this has to be set both in **[DEFAULT]** and again under **[sshd]**, otherwise it falls back to the default **iptables-multiport** action.

```
~$ sudo systemctl enable --now fail2ban
~$ sudo fail2ban-client status
~$ sudo fail2ban-client status sshd
```

```
Status for the jail: sshd
|- Filter
| |- Currently failed: 6
| |- Total failed:    22
| `-- File list:      /var/log/auth.log
`-- Actions
   |- Currently banned: 2
   |- Total banned:    2
   `-- Banned IP list: 62.171.168.158 193.176.251.229
```

Within minutes, fail2ban was already banning real attackers from the internet. Checking `firewalld` confirms the bans are applied there:

```
~$ sudo firewall-cmd --list-rich-rules
```

```
rule family="ipv4" source address="143.110.178.177" port port="ssh" protocol="tcp" reject type="deny"
rule family="ipv4" source address="31.179.197.26" port port="ssh" protocol="tcp" reject type="deny"
rule family="ipv4" source address="66.116.204.122" port port="ssh" protocol="tcp" reject type="deny"
rule family="ipv4" source address="142.93.181.239" port port="ssh" protocol="tcp" reject type="deny"
```

This shows a working setup, tested against real brute-force traffic and combined with `firewalld`'s default-reject policy and our key-only SSH login.

2.8. Backend Hardening & App's Security Configurations

2.8.1. Fixing the most obvious mistakes

From the beginning of the project, our team tried to pay attention to security features, but we made some common mistakes that we had to fix.

The only external API that is implemented is ORS . For it to work, there is a need for an API key. Initially, we just added this key directly inside the `appsettings.json` file. The big mistake that was made was the fact that the `appsettings.json` file was not added into `.gitignore`. Because of this, our private API key became public on GitHub. Once we noticed, the team immediately changed the key on the ORS platform, generated a new one, and added `appsettings.json` into `.gitignore` so it won't happen again.

Also, we had another bad practice where the JWT signing key was completely hardcoded inside the C# authentication code. We refactored this by moving the JWT secret into the `appsettings.json` configuration file for enhanced security.

In the `TourController`, we also noticed that anyone could access the endpoints without being logged in. We refactored this by adding the `[Authorize]` attribute at the class level to strictly limit access only to authenticated users who carry a valid token.

2.8.2. Database Layer Isolation and Docker Networking

When we first deployed the PostgreSQL database container, we did the most obvious thing: we opened the standard port in our `docker-compose.yml` file using ports: - "5432:5432". We quickly realized this was a major security issue because it meant anyone on the internet who scanned our Azure public IP (158.158.51.223) could try to brute-force their way directly into our database server.

To fix this, we removed public ports and we created two private networks. We completely deleted the ports section from the db service in Docker Compose. Now, the port 5432 is closed to the outside world. The two separate internal bridge networks at the bottom of our compose file: `frontend_net` and `backend_net`. We applied the `internal: true` flag to the database network.

The frontend container is only on `frontend_net`. The database container is strictly isolated inside `backend_net`. The .NET backend container has “one foot in each network” because it is added to both. This means the backend can still talk to the internet and the frontend, but the database is completely invisible from the outside and can only accept queries coming directly from our C# app container.

2.8.3. Principle of Least Privilege & SQL Role Hardening

One issue we found was in our connection string. The backend was connecting to PostgreSQL using the main administrator account (`Username=user`). This violates the “Principle of Least Privilege”. If a hacker found a vulnerability in our C# backend, they would automatically gain full admin rights over the database and could run a `DROP DATABASE` command.

We had some trouble setting up a separate user because our permissions were too locked down at first, but we managed to run a SQL command inside the running container using Docker terminal commands. We created a separate, limited user called `tourplanner_app` and gave it only the strict permissions needed for daily application use:

```
CREATE USER tourplanner_app WITH PASSWORD 'SecuredPassword2026!';
GRANT CONNECT ON DATABASE tourplanner_db TO tourplanner_app;
GRANT USAGE ON SCHEMA public TO tourplanner_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO tourplanner_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO
```

After creating this limited role, we updated our backend environment connection string in Docker Compose to use `Username=tourplanner_app`. However, we found that switching from the superuser to this limited user broke the application startup, as the restricted permissions prevented the backend from automatically creating the database tables. We ultimately had to revert to the superuser configuration to keep the project functional.

2.8.4. Media Upload Safeguards (Anti-DoS)

In our initial implementation of the `PhotosController`, the endpoint just accepted any file that the frontend sent. An attacker could easily upload a huge file to fill up our server’s disk space, or upload a malicious executable and try to execute it on our server.

To protect the server from storage exhaustion and Remote Code Execution , we implemented a validation block directly inside the `ImageStorageService`. We defined a constant `maxFileSize = 5 * 1024 * 1024` . If a file's length is larger than this, the code instantly throws an `BadRequest` and aborts, saving our server RAM and disk. We created an array of allowed extensions: `.jpg`, `.jpeg`, and `.png`. The code uses `Path.GetExtension()` on the uploaded file name, forces it to lowercase with `.ToLower()` (so attackers cannot bypass filters by using `.PNG`), and checks if it exists in our list using `Array.IndexOf()`. If the result is less than 0, the server rejects the file immediately.

2.8.5. Rate limiter and web redirection

To protect our .NET web server from DoS attacks and brute-force attempts, we initially attempted to implement an application-level rate limiter using the `Microsoft.AspNetCore.RateLimiting` middleware. We created a dedicated policy named `FixedWindowPolicy` to restrict clients to a maximum of 20 requests per minute on vulnerable endpoints, using the `Host` header as a partition key to handle Docker's internal networking. However, we ultimately abandoned this implementation and performed a git revert because the middleware introduced a breaking conflict within the security pipeline, consistently returning 401 Unauthorized errors during Bearer JWT validation. By tracing the security protocol step-by-step, we determined that this specific intersection is where the authentication logic breaks down.

We also attempted to implement HTTPS redirection using `app.UseHttpsRedirection()` to force traffic from port 8080 to a secure port. However, during frontend testing, this broke the authentication flow. While the CORS OPTIONS request succeeded (HTTP 204), the actual POST request triggered an HTTP 307 redirect. Because the local Linux Docker container lacked a trusted SSL/TLS certificate, the browser blocked the redirection immediately. Since we could not bypass this certificate validation error within our Docker environment, we ultimately dropped the HTTPS redirection and reverted the changes to keep the application functional.

```
tour-planner-be | [13:51:07 INF] Request starting HTTP/1.1 OPTIONS http://localhost:8080/api/a
tour-planner-be | [13:51:08 INF] CORS policy execution successful.
tour-planner-be | [13:51:08 INF] Request finished HTTP/1.1 OPTIONS http://localhost:8080/api/a
tour-planner-be | [13:51:08 INF] Request starting HTTP/1.1 POST http://localhost:8080/api/a
tour-planner-be | [13:51:08 INF] CORS policy execution successful.
tour-planner-be | [13:51:08 INF] Request finished HTTP/1.1 POST http://localhost:8080/api/a
```

2.9. Deployment of our current version of our under-development application

To finish this protocol, we will deploy our `TourPlanner` web-application in its current form (far from finished, but functional). The purpose of this is to test functionality and integration in our set up, while monitoring the integrity of our virtual machine.

Since our VM's resources are limited, but also to separate our execution environment from our development one following standard practices, we built the application locally and uploaded the binary files on DockerHub, pulling only the final images [20][22]:

Local:

```
docker compose build
docker compose push
```

Virtual Machine:

```
~$ cd /opt/app_folder
~$ sudo nano docker-compose.yml
```

Complete production docker-compose.yml:

```
services:
  db:
    image: postgres:15
    container_name: tour-planner-db
    restart: always
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: tourplanner_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d tourplanner_db"]
      interval: 5s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 200M
        reservations:
          memory: 150M
    networks:
      - backend_net
    logging:
      driver: syslog
```

```

        options:
            syslog-address: "udp://127.0.0.1:514"
            tag: "docker/{{.Name}}"

backend:
    image: armandveress/tourplanner-backend:latest
    container_name: tour-planner-be
    restart: on-failure
    depends_on:
        db:
            condition: service_healthy
    ports:
        - "8080:8080"
    volumes:
        - /opt/app_folder/photos:/app/Storage
        - ./logs:/app/logs
    environment:
        - ConnectionStrings__DefaultConnection=Host=db;Database=tourplanner_db;Username=
        - ASPNETCORE_ENVIRONMENT=Production
        - ASPNETCORE_HTTP_PORTS=8080
    deploy:
        resources:
            limits:
                memory: 250M
            reservations:
                memory: 200M
    networks:
        - backend_net
        - frontend_net
    logging:
        driver: syslog
        options:
            syslog-address: "udp://127.0.0.1:514"
            tag: "docker/{{.Name}}"

frontend:
    image: armandveress/tourplanner-frontend:latest
    container_name: tour-planner-fe
    ports:
        - "80:80"
    deploy:
        resources:

```

```

        limits:
            memory: 50M
        reservations:
            memory: 30M
    networks:
        - frontend_net
    logging:
        driver: syslog
        options:
            syslog-address: "udp://127.0.0.1:514"
            tag: "docker/{{.Name}}"

volumes:
    postgres_data:

networks:
    frontend_net:
        driver: bridge
    backend_net:
        driver: bridge
        internal: true

```

```

/opt/app_folder$ sudo docker compose pull
/opt/app_folder$ sudo docker compose up -d
[+] up 6/6
Network app_folder_backend_net Created
Network app_folder_frontend_net Created
Volume app_folder_postgres_data Created
Container tour-planner-db Healthy
Container tour-planner-fe Started
Container tour-planner-be Started

```

30 minutes check:

```

~$ sudo docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED
d892ccd58322	armandveress/tourplanner-backend:latest	"dotnet backend.dll"	30 minutes ago
068bd45327e0	postgres:15	"docker-entrypoint.s..."	30 minutes ago
1d32b7dc8906	armandveress/tourplanner-frontend:latest	"/docker-entrypoint..."	30 minutes ago

All images were pulled successfully and all containers started without errors. However, even though the containers were all running successfully on the virtual machine, the team noticed

the frontend under our public-ip was not reachable, returning an `ERR_CONNECTION_TIMED_OUT`. We have found a documented article about a similar case where this issue was encountered [21], caused by Azure NSG that was blocking web traffic due to missing port rules. In our case, we only had one port rule up until that point, allowing the team-members to connect on port 22 through SSH. The problem was fixed after adding a rule for port 80 and 8080.

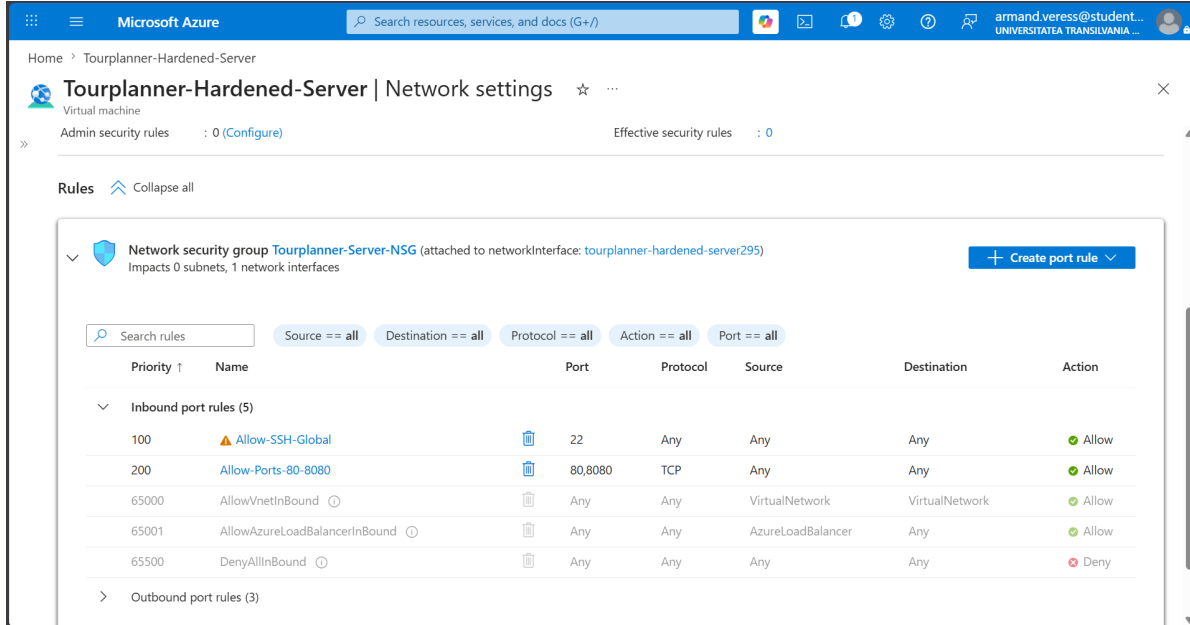


Figure 2: Figure 2. Azure NSG active port rules

After the app became reachable, we still needed to fix a few issues before it became functional:

- we added a production environment in our frontend to fix hardcoded `apiUrl` issues
- we updated the Cors policy to allow our public-ip

3. Conclusions

With our application up and running and the hand-in deadline approaching, we took a step back and reevaluated the current state of our system:

- At the core level, we have a minimal software selection designed for our container-based application. We configured swap memory and memory limits for all working containers, we isolated our most vulnerable volume using a `LoopDevice` with strict `noexec` restrictions and activated periodic updates for our operating system. Last but not least,

AppArmor enforces strict rules at our kernel level including a dedicated profile for our log-extraction service.

- At our Docker-application layer, we have isolated our database completely from the internet using internal bridge networks, we removed hardcoded secrets and applied media upload safeguards.
- At the network level, having used Azure Cloud infrastructure, the Network Security Group already blocks everything except our strictly required ports, on top of which we added `firewalld` as a second measure inside our operating system. `fail2ban` protects us against brute-force attacks and last but not least, we only have remote access through SSH keys.

So is our system secure?

If we look at the virtual machine's level and the operating system we would say rather yes than no. However, even though all the implemented measures protect the system's integrity quite well, we are aware that the current state of our TourPlanner application itself still has severe security holes.

Even so, with our application running on our somewhat hardened server, we consider successful our main goal of exploring the entire process of deploying an application on the internet, while employing several security configurations on all levels, meant to protect our system.

References

- Installing Docker:
 - [0] <https://docs.docker.com/engine/install/debian/>
- Securing Debian Manual:
 - [1] <https://www.debian.org/doc/manuals/securing-debian-manual/sec-services.en.html#idm2294>
- Swap memory considerations and configuration:
 - [2] <https://www.redhat.com/en/blog/cloud-swap>
 - [3] <https://docs.platform9.com/private-cloud-director/2025.10/getting-started/pre-requisites/configure-swap-ubuntu>
- Memory management Docker:
 - [4] <https://oneuptime.com/blog/post/2026-02-08-how-to-optimize-docker-for-memory-intensive-applications/view>

- Debian periodic updates:
 - [5] <https://wiki.debian.org/PeriodicUpdates>
- AppArmor Security Framework:
 - [6] https://www.reddit.com/r/debian/comments/19a060y/do_i_have_to_configure_apparmor/
 - [7] <https://wiki.debian.org/AppArmor/HowToUse>
 - [8] https://apparmor.net/about/distros/Distro_debian/#building
 - [9] <https://docs.docker.com/engine/security/apparmor/>
- Rsyslog network input modules:
 - [10] [<https://www.rsyslog.com/doc/master/configuration/modules/imudp.html>](<https://docs.rsyslog.com>)
- firewalld documentation:
 - [11] <https://firewalld.org/documentation/>
- fail2ban firewalld integration:
 - [12] [<https://github.com/fail2ban/fail2ban>](<https://github.com/fail2ban>)
- Automated container upgrades:
 - [13] <https://techroads.org/scheduling-a-docker-compose-image-auto-update-in-linux/>
- Logical Volume Manager (LVM):
 - [14] https://weihanglo.tw/debian-R-setup/doc/install_lvm.html
 - [15] <https://debian-handbook.info/browse/stable/advanced-administration.html#sect.lvm>
- LoopDevice:
 - [16] <https://ops.tips/blog/lvm-on-loopback-devices/>
 - [17] <https://www.lenovo.com/us/en/glossary/loop-device/>
 - [18] <https://man7.org/linux/man-pages/man4/loop.4.html>
- Resource Exhaustion Testing:
 - [19] <https://oneuptime.com/blog/post/2026-01-30-resource-exhaustion-testing/view>
- Deployment:

- [20] <https://docs.docker.com/build/building/best-practices/>
- `ERR_CONNECTION_TIMED_OUT` Issue: [21] <https://medium.com/@ritagitamo/how-i-deployed-a-full-stack-web-app-on-microsoft-azure-what-worked-what-broke-and-what-i-e6941faea79c>
- [22] <https://dev.to/mahmud-r-farhan/docker-deployment-for-beginners-a-simple-and-practical-guide-56e8>